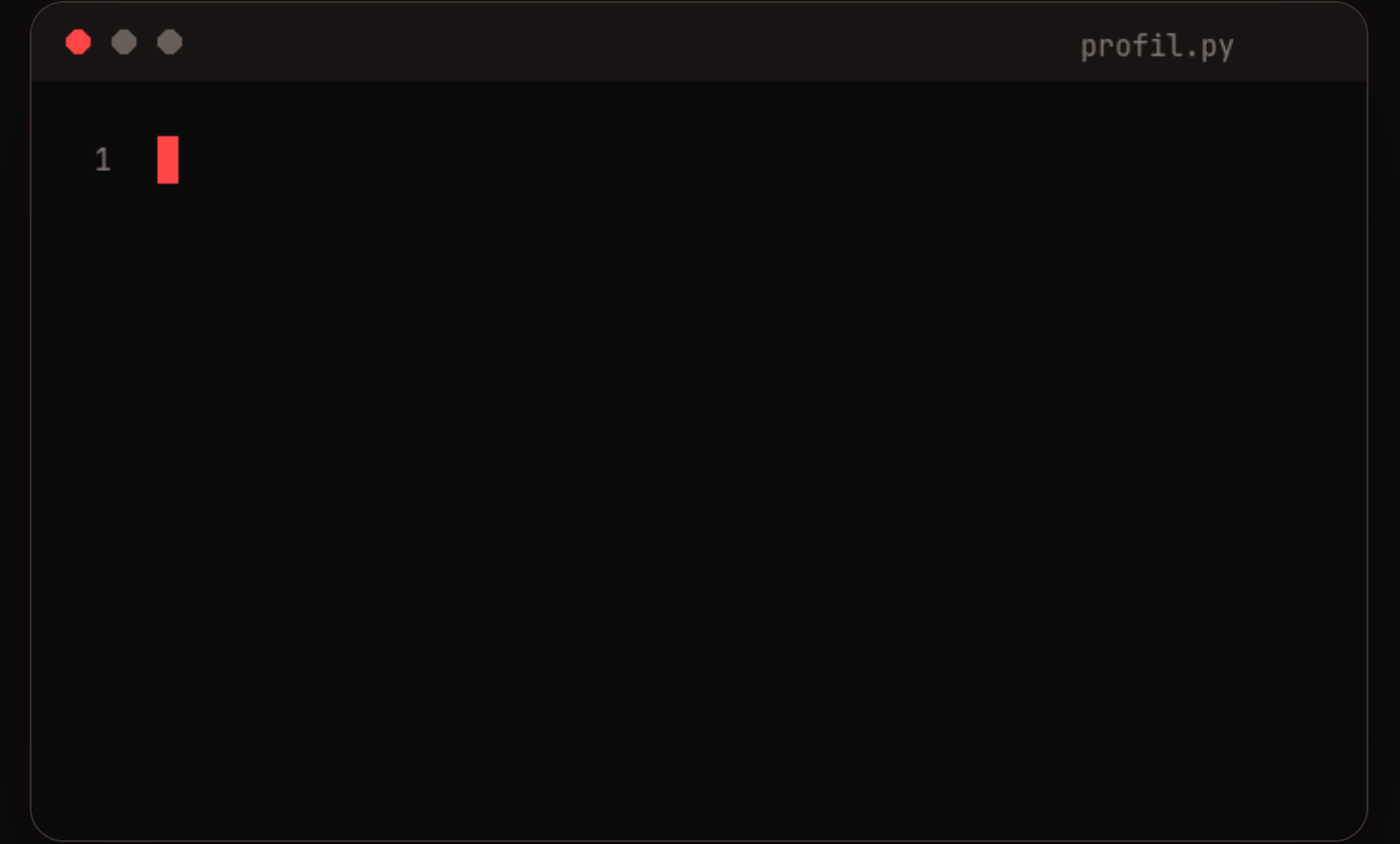


Güven sınırını geçmek

Tehdit modelleme ve enjeksiyon: girdinin koda
dönüştüğü an



BUGÜN

Bugünün akışı

- | | | |
|----|-------------------------|--|
| 01 | Kısa tekrar | Geçen haftanın üç fikri: güvenilmeyen girdi, güven sınırı, saldırganın sorusu. |
| 02 | Tehdit modelleme | Bir sistemi saldırganın gözünden çizmek: DFD ve STRIDE. |
| 03 | Enjeksiyonun kök nedeni | Veri ile kodun aynı kanalda birleşmesi. |
| 04 | SQL, komut, yol | Aynı hatanın üç yüzü ve tek yapısal savunma. |

TEKRAR

Geçen haftadan üç fikir

01

Güvenilmeyen girdi

Güven sınırını geçen her şey girdidir ve kanıtlanana kadar güvenilmezdir.

02

Güven sınırı

Kararlar sunucuda alınır; istemcide yapılan doğrulama isteğe bağlıdır.

03

İki soru

“Ne yapmalı?” testlerin sorusu. “Başka ne yapabilir?” saldırganın sorusu.

Bu hafta üçünü tek bir zafiyet sınıfında —enjeksiyon— birlikte göreceğiz.

BÖLÜM A

Tehdit modelleme

Kod yazmadan önce: bu sistemi kim, nereden, nasıl kırabilir?



YÖNTEM

Dört soru

SORU 1

Ne inşa ediyoruz?

Sistemi veri akış diyagramıyla (DFD) çiz: süreçler, depolar, aktörler, güven sınırları.

SORU 2

Ne ters gidebilir?

Her sınırı geçen okta STRIDE'ı uygula: hangi tehdit sınıfı mümkün?

SORU 3

Ne yapacağız?

Her tehdide bir karşı önlem; kabul edilen riski açıkça yaz.

SORU 4

İyi iş çıkardık mı?

Modeli kod ve testlerle karşılaştır; sistem değişince modeli güncelle.

Adam Shostack'ın dört sorusu. Model bir kez değil, sistem değiştikçe yenilenir.

Altı tehdit sınıfı

Harf	Tehdit	Kırdığı ilke	Örnek
S	Spoofing (kimlik taklidi)	Kimlik doğrulama	Başkasının oturumuyla giriş
T	Tampering (kurcalama)	Bütünlük	Fiyatı ya da rolü değiştirmek
R	Repudiation (inkâr)	İzlenebilirlik	İşlemi yapmadım demek, log yok
I	Information disclosure	Gizlilik	Veri sızıntısı
D	Denial of service	Erişilebilirlik	Servisi durdurmak
E	Elevation of privilege	Yetkilendirme	Kullanıcıdan yöneticiye çıkmak

Geçen haftanın kodunu modelleyelim

```
# güven sınırı: tarayıcı →|→ sunucu
```

```
GET /profil?id=1
```

```
# süreç: profil()
```

```
# depo: kullanıcılar tablosu
```

```
# akış: id → SQL sorgusu → DB
```

Tampering + Information Disclosure

id alanı bir güven sınırını geçiyor ve doğrudan sorguya giriyor. STRIDE'in T ve I harfleri tam da burada. Kök neden hep aynı: veri, koda dönüşüyor.

Aynı diyagram komut satırında, dosya yollarında ve şablon motorlarında da tekrar eder.

BÖLÜM B

Enjeksiyon

OWASP Top 10 · A05:2025 — hâlâ en yaygın ve en ölümcül sınıflardan biri.



Tek bir kök neden: veri ile kod aynı kanalda.

Uygulama bir metin kuruyor; metnin bir kısmı sizin kodunuz, bir kısmı kullanıcının verisi. Yorumlayıcı ikisini ayırt edemez. Savunma kaçış karakteri eklemek değil, kanalları ayırmaktır.



Aynı hata, farklı yorumlayıcı

SQL

Veritabanı

Sorgu metnine giren girdi — CWE-89.

OS

Kabuk / komut

Komut satırına giren girdi — CWE-78.

PATH

Dosya sistemi

Yola giren girdi (../) — CWE-22.

TEMPLATE

Şablon motoru

İfade olarak yorumlanan girdi —
CWE-1336.

NoSQL

Belge sorgusu

Operatör enjekte eden girdi — CWE-943.

XML

Ayrıştırıcı

Harici varlık (XXE) — CWE-611.

SQL

Girdi sorguya dönüşünce

```
uid = request.args.get("id")          # güvenilmeyen
q = "SELECT ad FROM kullanicilar "
q += "WHERE id = " + uid               # veri, sorgu metnine yapıştı
db.execute(q)
```

Beklenen

id = 1 → tek kullanıcı döner.

Girdi değişince

id, sorgunun mantığını değiştirecek şekilde seçilirse tek satır yerine tablonun tamamı dönebilir.

Enjeksiyon nasıl görünür

Tür	Fikir	Belirti
In-band	Sonuç doğrudan yanıtta görünür (UNION, hata mesajı).	Sayfada fazladan veri
Blind (boolean)	Yanıt doğru/yanlışına göre değişir.	İçerik var/yok
Blind (zaman)	Koşullu gecikme ile bit bit okuma.	Yavaş yanıt
Second-order	Girdi saklanır, sonra başka sorguda tetiklenir.	Gecikmeli etki

Sonuç doğrudan yanıtta: UNION

```
-- kategori değeri doğrudan sorguya giriyor
SELECT ad, fiyat FROM urun WHERE kategori = 'GIRDI'

-- iki sütunlu sorguya eşit sütunlu bir UNION eklenirse:
-- ' UNION SELECT kullanıcı, parola FROM hesap --
-- yanıt artık ürünlerle birlikte hesap tablosunu da döker
```

Neden çalışır

Girdi tırnaktan çıkıp sorgunun gövdesine karışıyor; UNION iki sonuç kümesini birleştirir.

Ön koşul

Sütun sayısı ve tipleri uymalı; saldırgan bu yüzden önce sütun sayısını arar.

Yanıt görünmese de sızar

```
-- 1) Boolean: yanıt doğru/yanlış'a göre değişir
id=1 AND (SELECT SUBSTR(parola,1,1) FROM hesap LIMIT 1)='a'
--    sayfa normal → harf doğru ; boş → yanlış

-- 2) Zaman tabanlı: koşul doğruysa yanıt gecikir
id=1 AND IF(SUBSTR(parola,1,1)='a', SLEEP(3), 0)
```

Yanıta hiç veri görünmese bile bilgi bit bit çıkarılabilir; “çıktı yok” güvenlik değildir.

ORM tek başına korumaz

```
# Django – ham SQL'de string birleştirme yine açık
Kullanici.objects.raw("... WHERE ad = '%s'" % ad)      # x
Kullanici.objects.raw("... WHERE ad = %s", [ad])       # ✓

# SQLAlchemy – text() içinde de aynı kural
session.execute(text("... WHERE ad = '" + ad + "'"))  # x
session.execute(text("... WHERE ad = :ad"), {"ad": ad}) # ✓
```

ORM'in normal sorgu kurucusu güvenlidir; tehlike `.raw()/extra()/text()` ile metne döndüğünüz yerdedir.

Savunma: hazır ifade (prepared statement).

Sorgunun iskeleti sabit derlenir; kullanıcı verisi ayrı bir kanaldan parametre olarak gider. Yorumlayıcı artık veriyi kod olarak okuyamaz — sınıfın tamamı kapanır.

SQL

Kanalları ayırmak

```
# x veri koda karışıyor
q = "... WHERE id = " + uid

# ✓ iskelet sabit, veri parametre
cur.execute(
    "... WHERE id = %s", (uid,))
```

Aynı ilke her dilde var: sorgu metnine değil, ayrı parametre kanalına yaz.

Her dilde aynı ilke

```
// Java – PreparedStatement
var ps = c.prepareStatement("... WHERE id = ?");
ps.setInt(1, id);

// C# – parametrelili komut
cmd.CommandText = "... WHERE id = @id";
cmd.Parameters.AddWithValue("@id", id);

// PHP (PDO) ve Node (pg)
$pdo->prepare("... WHERE id = ?")->execute([$id]);
client.query("... WHERE id = $1", [id]);
```

Yer tutucu farklı (?, @id, \$1) ama fikir tek: veri asla sorgu metni olmaz.

Parametrelenemeyeni izin listesiyle bağla

```
# ✗ sıralama sütunu doğrudan girdiden
q = f"SELECT * FROM urun ORDER BY {kolon}"    # kolon = ?

# ✓ kolon/tablo adı parametre olamaz → izin listesi
IZIN = {"ad", "fiyat", "tarih"}
if kolon not in IZIN:
    raise ValueError('gecersiz kolon')
q = f"SELECT * FROM urun ORDER BY {kolon}"    # artık güvenli
```

Tablo adları, kolon adları, ORDER BY yönü parametrelenemez; hepsini sabit bir kümeye eşle.

KOMUT

Kabuğa giden girdi

```
# ✗ kabuk metni kuruluyor
os.system("ping -c1 " + host)          # host saldırganın

# ✓ argümanlar dizi olarak, kabuk yok
subprocess.run(["ping", "-c1", host], shell=False)
```

Kök neden

shell=True kabuğu araya sokar; kabuk metni ayrıştırır ve girdi komuta dönüşür.

Kalıcı risk

Kanal ayrılrsa bile argüman enjeksiyonu (-o, -- ile başlayan girdi) sürebilir; girdiyi yine doğrula.

KOMUT • ARGÜMAN

Kanal ayrıldı, risk bitmedi

```
# kabuk yok ama girdi hâlâ bir ARGÜMANA dönüşüyor
subprocess.run(["git", "log", kullanıcı_girdisi])
# girdi "--output=/etc/cron.d/x" olabilir → seçenek enjeksiyonu

# ✓ değerleri -- ile ayır ve biçimi doğruyla
subprocess.run(["git", "log", "--", dosya])
```

Argüman enjeksiyonu (CWE-88): girdi komut değil ama bir bayrak gibi yorumlanır. -- ayırıcı ve allowlist şart.

YOL

Dosya yoluna giden girdi

```
# ✗ kullanıcı adı yola yapışıyor
open("/veri/" + ad)                                # ad = "../../etc/..." olabilir

# ✓ önce kanonikleştir, sonra kök içinde mi diye doğrula
tam = os.path.realpath(os.path.join(KOK, ad))
assert tam.startswith(KOK + os.sep)
```

Kural: birleştir → kanonikleştir (realpath) → sınırı doğrula. Uzantı ve içerik kontrolü de eklenir.

Girdi tipi değişince koşul çöker

```
// beklenen: iki string ile eşleşme
db.hesap.find({ kullanıcı: girdiK, parola: girdiP })

// girdiP bir NESNE olursa: { "$ne": null }
//   → parola ≠ null her zaman doğru   → kimlik atlatma

// ✓ girdinin string olduğunu doğrula
if (typeof girdiP !== "string") return reddet();
```

NoSQL'de de kök neden aynı: veri (string) bir operatöre (kod) dönüşüyor. JSON gövdesinde tip doğrulaması şart.

SSTI

Girdi şablonun kendisi olmamalı

```
# x kullanıcı verisi ŞABLON metnine giriyor
render_template_string("Merhaba " + ad)

# ad = {{7*7}} → ekranda 49 → motor girdiyi çalıştırdı

# ✓ veri, şablona PARAMETRE olarak girer
render_template("selam.html", ad=ad) # {{ ad }} yalnızca metin
```

SSTI (CWE-1336)

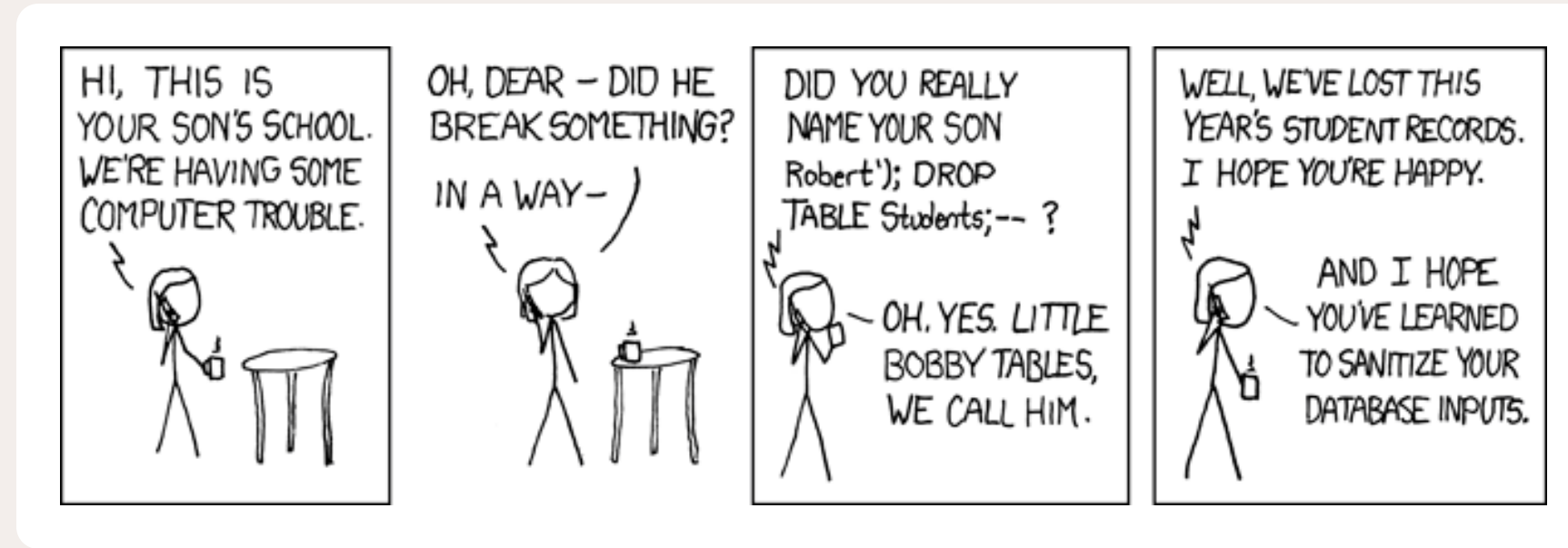
Girdi ifade olarak yorumlanırsa uzaktan kod çalıştırmaya kadar gider.

XXE (CWE-611)

XML'de de aynı sınıf: harici varlıkları ve DTD'yi kapat; ayrıştırıcı varsayılanı güvensiz.

KLASİK

xkcd 327 — “Little Bobby Tables”



Okul öğrencinin adını doğrudan sorguya koymuş. Espri gerçek: girdi her zaman veri kalmalı.

VAKA

MOVEit Transfer • CVE-2023-34362

Ne oldu

Dosya transfer yazılımında bir SQL enjeksiyonu; oradan web shell ve toplu veri sızıntısı. ClOp, yüzlerce kurumu vurdu.

Dersle bağ

Kök neden bu haftanın konusu: girdi sorguya dönüştü. Prepared statement sınıfı kapatıldı.

Dönem projesi tam da bunu ister: kök neden, sömürü yolu, yama eleştirisi. İyi bir aday.

ÖZET

Bir cümlede

KÖK NEDEN

Aynı kanal

Veri ile kod tek metinde birleşiyor.

SAVUNMA

Kanalı ayır

Parametre, argüman dizisi, kanonik yol, güvenli şablon.

TASARIM

En az yetki

Kırılsa bile hasar dar kalsın.

HAFTAYA

Haftaya kadar

PortSwigger

Web Security Academy'de SQL injection ve OS command injection laboratuvarlarından birer tanesini bitirin.

Kendi ortamınız

DVWA ya da Juice Shop'u Docker'da çalıştırın; istekleri Burp'ten izleyin.

Haftaya: tarayıcı güvenlik modeli, XSS ve CSRF. Etik: yalnızca kendi laboratuvarınızda.



SORU • CEVAP

Sorular?

